

From Code to Mathematics: The Unified Particle Filter in `pfilter.jl`

Debsurya

September 8, 2026

Abstract

This document gives a line-by-line mathematical reading of the unified particle filter implemented in `src/pfilter.jl` of the `PartiallyObservedMarkovProcesses.jl` package. The filter generalizes the classical bootstrap particle filter along two axes: an effective-sample-size *trigger* that governs whether resampling occurs at a given observation time, and a tempering parameter *target* (β) that governs how much of a particle’s weight is discharged into selection probability versus retained on the surviving particle. We derive the algorithm from first principles and show that the two extreme settings recover the classical bootstrap filter and the fully weighted (never-resampling) filter respectively. Three genuine defects turned up in the course of this derivation and were corrected in the code. An earlier translation of the tempered resampling step indexed the retained weight by selection slot rather than by selected ancestor (Section 2), and the terminal ancestry draw used to report a simulated trajectory sampled uniformly over terminal particle indices rather than in proportion to their weight (Section 6). The third, and the principal subject of this document, is that power-tempered resampling discards a normalizing constant C whenever $0 < \beta < 1$: Section 3 derives C exactly, from the same importance-sampling argument used for the other two defects, shows by an exact two-particle calculation that dropping it biases the running log likelihood low by $\Theta(1/n)$ per resampling step, and confirms the bias with an end-to-end filtering test built specifically to be sensitive to it. This finding was itself derived once before, implemented, and then wrongly retracted on the strength of an underpowered numerical check; Section 8 narrates that episode and draws the general lesson about what a null Monte Carlo comparison can and cannot show. Section 7 gives the general validation of the filter against the exact Kalman likelihood for the Gompertz model.

Contents

1	The POMP model and the bootstrap particle filter	2
1.1	The partially observed Markov process	2
1.2	The bootstrap particle filter	3
2	Power-tempered systematic resampling	4
2.1	Tempering the resampling step	4
2.2	The four-argument <code>systematic_resample!</code>	4
3	The normalizing constant discarded at partial resampling	7
3.1	The telescoping identity	7
3.2	Proper weighting: a sufficient condition	7
3.3	The normalizing constant, exactly	8
3.4	Conditional mean one is not enough	9

3.5	An exact enumeration, and an end-to-end test	10
3.6	The code	10
4	ESS-triggered resampling	11
4.1	Effective sample size	11
4.2	<code>compute_ess_logLik!</code> : the two methods	11
4.3	The trigger	12
5	The filtering step	12
5.1	Classic path: <code>trigger = 1</code> , <code>target = 0</code>	13
5.2	General path: weighted, <code>trigger/target</code>	13
6	Weighted terminal ancestry draw	14
6.1	The problem	14
6.2	A two-particle counterexample	15
6.3	The corrected, five-argument <code>trace_ancestry!</code>	15
7	The Gompertz–Kalman validation	16
7.1	The log-Gompertz linear-Gaussian reduction	16
7.2	The scalar Kalman recursion	17
7.3	The Jacobian correction	17
7.4	Mapping the test	18
8	History: a normalizing constant lost, retracted, and reinstated	19
8.1	The retraction, and why it was wrong	19
8.2	What each instrument can see	20
8.3	The lesson	20

1 The POMP model and the bootstrap particle filter

1.1 The partially observed Markov process

A partially observed Markov process (POMP) consists of a latent Markov process $\{X_t\}$ together with a sequence of noisy observations $\{Y_n\}_{n=1}^N$ made at times $t_1 < t_2 < \cdots < t_N$. Write $x_{0:N} = (x_0, x_1, \dots, x_N)$ for a realization of the latent process, sampled at $t_0 < t_1 < \cdots < t_N$, and $y_{1:N} = (y_1, \dots, y_N)$ for the data. The model is specified by three components:

- an *initial density*, from which X_0 is drawn: $X_0 \sim r_{\text{init}}(\cdot; \theta)$;
- a *process transition density* $f(x_n \mid x_{n-1}; \theta)$, giving the Markov transition law from t_{n-1} to t_n ;
- a *measurement density* $g(y_n \mid x_n; \theta)$, giving the conditional law of the observation at t_n given the latent state there, with observations conditionally independent given the latent path.

The likelihood of the data under parameter θ is the marginal density

$$\mathcal{L}(\theta) = \int \cdots \int r_{\text{init}}(x_0; \theta) \prod_{n=1}^N f(x_n \mid x_{n-1}; \theta) g(y_n \mid x_n; \theta) dx_0 dx_1 \cdots dx_N,$$

an integral of dimension proportional to N that is analytically intractable for essentially every model of interest beyond the linear-Gaussian case. In the code, r_{init} , the process model, and g are supplied as the basic components `rinit`, `rprocess`, and `logdmeasure`; the package requires only that one can *simulate* from r_{init} and from $f(\cdot | x_{n-1}; \theta)$, and *evaluate* $g(y_n | x_n; \theta)$ pointwise — the plug-and-play property that motivates the sequential Monte Carlo approach below.

1.2 The bootstrap particle filter

The bootstrap particle filter (Gordon, Salmond & Smith, 1993) estimates $\mathcal{L}(\theta)$ by propagating a swarm of J particles $\{X_n^{1:J}\}$ through the filtering recursion, using the process model itself as the proposal (importance) distribution. Initialize $X_0^j \sim r_{\text{init}}(\cdot; \theta)$, $j = 1, \dots, J$, independently. At each observation time $n = 1, \dots, N$:

1. **Prediction.** Propagate each particle forward through the process model: $X_n^j \sim f(\cdot | X_{n-1}^j; \theta)$, independently across j .
2. **Weighting.** Assign each predicted particle an importance weight equal to the measurement density evaluated at the data:
$$w_n^j = g(y_n | X_n^j; \theta).$$
3. **Resampling.** Draw J indices $\xi_n^1, \dots, \xi_n^J$ with $\Pr(\xi_n^k = j) \propto w_n^j$, and set the filtered particle $X_n^k \leftarrow X_n^{\xi_n^k}$ for each k , discarding the weights (each surviving particle now carries equal weight $1/J$).

Because the process model is used as the proposal, the importance weight is exactly the measurement density — no Radon-Nikodym correction between proposal and target transition kernels is needed. The conditional likelihood contributed at time n (the density of y_n given $y_{1:n-1}$) is estimated, unbiasedly, by the empirical mean of the weights:

$$\hat{\mathcal{L}}_n = \frac{1}{J} \sum_{j=1}^J w_n^j,$$

and, since $\mathcal{L}(\theta) = \prod_{n=1}^N \mathcal{L}_n(\theta)$ by the Markov property, the overall likelihood is estimated by the product

$$\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N \hat{\mathcal{L}}_n = \prod_{n=1}^N \left(\frac{1}{J} \sum_{j=1}^J w_n^j \right).$$

This estimator is unbiased for $\mathcal{L}(\theta)$ (Del Moral, 2004) — a non-trivial fact, since it is a product of ratios of dependent random quantities, not simply an average of independent terms. The remainder of this document is concerned with a single question: which modifications of steps 1–3 above preserve this unbiasedness property, and what bookkeeping does the implementation require to make sure they do.

The package works throughout on the log scale, both for numerical stability (weights routinely span many orders of magnitude) and because the overall log likelihood decomposes as a sum,

$$\log \hat{\mathcal{L}}(\theta) = \sum_{n=1}^N \log \hat{\mathcal{L}}_n,$$

of the *conditional log likelihoods* — exactly the quantity accumulated, observation time by observation time, in the array `cond_logLik` in `pfilter.jl`.

2 Power-tempered systematic resampling

2.1 Tempering the resampling step

The classical resampling step (step 3 above) discharges the entire weight w^j of a particle into its selection probability. The unified filter instead factors each weight as

$$w = w^{1-\beta} \cdot w^\beta, \quad \beta \in [0, 1],$$

and uses only the first factor to drive selection: particle j is selected with probability proportional to $(w^j)^{1-\beta}$, and each selected particle *retains*, rather than discards, the second factor $(w^j)^\beta$ as its new weight — renormalized, as described below, to have unit mean over the particle swarm. The parameter β is called **target** in the code (and denoted β throughout this document to keep the two roles — the resampling parameter and the code identifier for it — visually distinct).

- At $\beta = 0$: selection probability $\propto w^1 = w$, retained weight $\propto w^0 = 1$. This is exactly the classical bootstrap resampling step: particles are chosen proportional to their full weight and emerge with equal weight.
- At $\beta = 1$: selection probability $\propto w^0 = 1$, retained weight $\propto w^1 = w$. Selection is now *uninformative* — every particle is equally likely to be chosen regardless of its weight — while the full weight is carried forward unchanged on the selected copy. No information is discarded; this is resampling in name only, and is used to reshuffle particle identity (for bookkeeping of ancestry) without touching the weighted representation.
- At intermediate β : a genuine interpolation. Some of the weight’s information is spent reducing the variance of the particle swarm’s spatial distribution (via selection), and the remainder is preserved as an informative weight on the surviving particles, to be combined multiplicatively with weights accrued at future observation times.

2.2 The four-argument `systematic_resample!`

The tempered resampling step is implemented in `src/pfilter.jl`, lines 457–491:

Listing 1: `systematic_resample!`, power-tempered systematic resampling (`pfilter.jl`, lines 457–491)

```
1 systematic_resample!(  
2     p::AbstractArray{I,1},  
3     w::AbstractArray{W,1},  
4     ucum::AbstractArray{W,1},  
5     beta::W,  
6 ) where {I,W} = begin  
7     @assert length(ucum)==length(w)==length(p)  
8     s::W = 0  
9     alpha = 1-beta  
10    @inbounds for j in eachindex(w)  
11        s += w[j]^alpha  
12        ucum[j] = s  
13    end  
14    n::I = length(w)  
15    i::I = 1  
16    du::W = s/n
```

```

17 u::W = -du*rand(W)
18 @inbounds for j in eachindex(p)
19     u += du
20     while (u > ucum[i] && i < n)
21         i += 1
22     end
23     p[j] = i
24 end
25 stot::W = s
26 @inbounds for j in eachindex(p)
27     ucum[j] = w[p[j]]^beta
28 end
29 @inbounds for j in eachindex(w)
30     w[j] = ucum[j]
31 end
32 m::W = mean(w)
33 w ./= m # subsequent steps rely on the weights having unit mean
34 log(m*stot/n)
35 end

```

We map this line by line onto the mathematics of systematic resampling (Kitagawa, 1996), the standard low-variance alternative to multinomial resampling.

Cumulative sum of tempered weights (lines 466–469). Writing $w^{1:J}$ for the incoming (linear, not necessarily unit-mean) weight vector and $\alpha = 1 - \beta$, the code computes the cumulative sums

$$S'_j = \sum_{i=1}^j (w^i)^\alpha, \quad j = 1, \dots, J, \quad S' := S'_J = \sum_{i=1}^J (w^i)^\alpha.$$

`ucum[j]` holds S'_j ; `s` accumulates to S' at the end of the loop.

The systematic sweep (lines 470–480). A single uniform random offset $u_0 \sim \text{Unif}(0, S'/J)$ is drawn (line 473, as `-du*rand(W)`), so that the first increment lands at exactly this offset), and J equally spaced points

$$u_k = u_0 + k \cdot \frac{S'}{J}, \quad k = 1, \dots, J,$$

are swept against the cumulative sum $S'_{1:J}$: the ancestor index for selection k is

$$p_k = \min\{j : S'_j \geq u_k\}.$$

This is systematic resampling on the tempered weights $(w^j)^{1-\beta}$: each ancestor j is selected with multiplicity concentrated near $J(w^j)^{1-\beta}/S'$, and, unlike independent multinomial draws, the selection has minimal variance subject to these marginal selection probabilities — a single sorted sweep rather than J independent draws. `p[j]` on exit is the array $\xi^{1:J} = (p_1, \dots, p_J)$ of selected ancestor indices; `stot` (line 481) holds S' itself, carried forward for use in the normalizing constant computed at the end of the routine (Section 3).

Ancestor-indexed retained weight (lines 482–484). This is the step at the mathematical heart of the tempered scheme, and the one most easily gotten wrong (Section 6 below describes an analogous error in ancestry tracing). Each original particle’s weight factored as $w^j = (w^j)^{1-\beta}(w^j)^\beta$;

the first factor has already been fully discharged into the selection step above (a particle with a large $(w^j)^{1-\beta}$ is selected many times). The remaining factor, $(w^j)^\beta$, is the piece of information not yet used, and it belongs to the *particle*, not to the *selection slot* it lands in. Consequently the retained weight of the k -th selected particle must be indexed by its ancestor:

$$\tilde{w}_k = (w^{p_k})^\beta, \quad k = 1, \dots, J,$$

exactly what line 483 computes: `ucum[j] = w[p[j]]^beta`. (`ucum` is reused here purely as scratch space, unrelated to its earlier role as the cumulative-sum array.) The alternative — raising the weight vector to the power β *before* indexing by `p`, i.e. computing $w[j] \leftarrow w[j]^\beta$ in place and then permuting — assigns to selection slot k the quantity $(w^k)^\beta$, the retained-weight power of whichever particle originally occupied position k , which is in general unrelated to the particle that was actually selected there. This positional form was the form found in an earlier translation of the algorithm, at Aaron King’s commit 1610d34 (see the memo in this directory, item 2), and has since been fixed there independently, at commit 1d8dbc0, in a form equivalent to the one below. It targets the wrong measure whenever $\beta > 0$, as Section 6 demonstrates concretely by a two-particle example in the closely analogous setting of the terminal ancestry draw.

Writing back and renormalizing (lines 485–489). The retained weights computed in the preceding loop are written into `w` (lines 485–487), and `w` is then divided by its own realized mean

$$m = \frac{1}{J} \sum_k \tilde{w}_k,$$

computed at line 488 (`m:W = mean(w)`) and applied at line 489 (`w ./= m`), so that on exit `w` has unit mean over the J particles. This is an implementation convention, not a mathematical necessity: the routine could equally well leave the retained weights unnormalized. The choice to renormalize is made because *every other stage of the filter* — in particular `compute_ess_logLik!`, discussed in Section 4 — assumes an entering weight vector with unit mean, so that the log of its arithmetic mean, needed for the conditional log likelihood, can be read off directly. Keeping this invariant globally is what makes it possible for the filter to treat “just resampled” and “several steps since the last resampling” states identically at every subsequent step.

The normalizing constant, returned (line 490). Renormalizing to unit mean divides every retained weight — and with it the swarm’s total mass — by the common factor m . Because the sampling measure driving selection was $q_j \propto (w^j)^{1-\beta}$ rather than the weights themselves, this renormalization removes real likelihood-scale information, not merely a bookkeeping convenience: Section 3 shows that the quantity

$$C = \frac{m \cdot S'}{J}$$

is exactly the resampling step’s own contribution to the normalizing constant of the filter’s target measure, that it has conditional mean one but is not independent of what it multiplies, and that dropping it biases the running log likelihood. The routine accordingly returns $\log C = \log(m \cdot \text{stot}/n)$ (line 490, with `n` the particle count J) rather than `nothing`, and the caller (Section 5, line 315) adds this return value into the running log likelihood.

3 The normalizing constant discarded at partial resampling

3.1 The telescoping identity

Consider two consecutive observation times, with the filter having just computed, at time $n - 1$, a properly weighted particle representation of the filtering distribution: a swarm $\{(X_{n-1}^j, \omega_{n-1}^j)\}_{j=1}^J$ with $\mathbb{E}\left[\frac{1}{J} \sum_j \omega_{n-1}^j h(X_{n-1}^j)\right] = \mathbb{E}[h(X_{n-1}) \mid y_{1:n-1}]$ for every test function h , and normalized so $\frac{1}{J} \sum_j \omega_{n-1}^j = 1$ (unit mean). Propagating each particle through the process model and weighting by the measurement density at time n produces an updated properly weighted swarm for the joint smoothing distribution of X_n given $y_{1:n}$, with un-normalized weights $\omega_{n-1}^j \cdot g(y_n \mid X_n^j; \theta)$, and — because the incoming swarm already had unit mean and represents the filtering measure correctly — the conditional likelihood contribution is exactly the mean of these updated weights:

$$\mathcal{L}_n = \frac{1}{J} \sum_{j=1}^J \omega_{n-1}^j g(y_n \mid X_n^j; \theta).$$

This is the content of the telescoping (or "sequential") identity $\mathcal{L}(\theta) = \prod_n \mathcal{L}_n(\theta)$ specialized to a weighted particle representation: *as long as the entering weights have unit mean and properly represent the filtering measure, the conditional likelihood at each step is recovered simply by averaging the updated, un-normalized weights* — no correction term is needed for the propagate/weight half of the recursion. The delicate step is what happens next, at resampling: the identity survives only if the swarm handed to the next observation time remains a properly weighted representation of the same measure, up to a normalizing constant that must be tracked, not discarded. The rest of this section derives that constant exactly and shows what goes wrong when it is dropped.

3.2 Proper weighting: a sufficient condition

After weighting, resampling replaces the properly weighted swarm $\{(X_n^j, w_n^j)\}$ — where, dropping the accumulated-weight notation ω used above, w_n^j denotes the current, possibly un-normalized weight at time n — with a new swarm $\{(X_n^{p_k}, \tilde{w}_n^k)\}_{k=1}^J$ obtained via power-tempered systematic resampling with parameter β . One sufficient condition for the telescoping identity to continue to hold undisturbed at the next step is that the new swarm remain *properly weighted* for the same filtering measure in the classical importance-sampling sense: for every test function h ,

$$\mathbb{E}\left[\frac{1}{J} \sum_{k=1}^J \tilde{w}_n^k h(X_n^{p_k}) \mid X_n^{1:J}, w_n^{1:J}\right] = \frac{1}{S} \sum_{j=1}^J w_n^j h(X_n^j), \quad S := \sum_{j=1}^J w_n^j,$$

i.e. the resampled swarm remains an unbiased (times a known constant) representation of the same weighted empirical measure. A direct calculation — using $\Pr(p_k = j) = (w_n^j)^{1-\beta}/S'$ with $S' = \sum_i (w_n^i)^{1-\beta}$, and retained weight $\tilde{w}_n^k = (w_n^{p_k})^\beta/m$ where m is whatever normalizing constant is applied — shows

$$\mathbb{E}\left[\tilde{w}_n^k h(X_n^{p_k}) \mid X_n^{1:J}, w_n^{1:J}\right] = \frac{1}{m} \sum_{j=1}^J \frac{(w_n^j)^{1-\beta}}{S'} (w_n^j)^\beta h(X_n^j) = \frac{1}{m S'} \sum_{j=1}^J w_n^j h(X_n^j).$$

Comparing with the target above, this sufficient condition holds exactly when $m S' = S$, i.e. when the retained weight of the k -th selected particle is taken to be

$$\tilde{w}_n^k = (w_n^{p_k})^\beta \cdot \frac{S'}{S},$$

which, when the incoming weights already have unit mean ($S = J$), is $(w_n^{p_k})^\beta \cdot S'/J$ — this is the correct, and the only, assignment that makes the resampled swarm properly weighted in the sense above. It is not, however, the assignment the code makes, and it is worth being precise about the gap between the two before asking whether that gap actually matters.

3.3 The normalizing constant, exactly

Rather than the properly weighted assignment $\tilde{w}_n^k = (w_n^{p_k})^\beta \cdot S'/S$ of Section 3.2, the code renormalizes to *unit mean*, dividing every retained weight $(w_n^{p_k})^\beta$ by the realized sample mean

$$m = \frac{1}{J} \sum_{k=1}^J (w_n^{p_k})^\beta.$$

Both are valid ways to fix the overall scale of the resampled swarm, and they generally disagree: S/S' is a deterministic function of the pre-resampling weights alone, while m is a realized average over the J systematic selections. Writing $R_k = w_n^{p_k} / (J q_{p_k})$ with $q_j = (w_n^j)^{1-\beta} / S'$ for the importance weight the proper-weighting argument assigns to selection k — so that $R_k = (S'/J) (w_n^{p_k})^\beta$ — the code's realized weight is $V_k = (w_n^{p_k})^\beta / m$, related to the properly weighted R_k by

$$V_k = \frac{R_k}{C}, \quad C := \frac{m S'}{J} \quad (\text{using } S = J \text{ under the incoming unit-mean convention}).$$

C is exactly the resampling step's own contribution to the normalizing constant of the unnormalized (Feynman–Kac) measure the filter is estimating: it is the sample mean of the properly weighted quantities R_k themselves,

$$C = \frac{1}{J} \sum_{k=1}^J R_k,$$

as a short calculation confirms: $\frac{1}{J} \sum_k R_k = \frac{1}{J} \sum_k (S'/J) (w_n^{p_k})^\beta = (S'/J) \cdot m$, matching the definition above. This is not a bookkeeping curiosity — C is a piece of the likelihood, on exactly the same footing as the mean weight computed at every other observation time in `compute_ess_logLik!` (Section 4) — and renormalizing $V_k = R_k/C$ to unit mean, as the code does, silently divides it out. Proposition 1 below records exactly when C is forced to 1 and when it is not.

Proposition 1. *Suppose the incoming weight vector $w^{1:J}$ has unit mean, $\frac{1}{J} \sum_j w^j = 1$ (as guaranteed by `compute_ess_logLik!`, Section 4, on entry to resampling). Let $S' = \sum_j (w^j)^{1-\beta}$ and let m be the realized mean, over the J systematic selections $p_{1:J}$, of $(w^{p_k})^\beta$, and $C = m S' / J$. Then:*

- (a) $C = 1$ identically (for every realization of the resampling randomness) when $\beta = 0$;
- (b) $C = 1$ identically when $\beta = 1$;
- (c) $\mathbb{E}[C \mid w^{1:J}] = 1$ exactly, for every $\beta \in [0, 1]$ and every realization of the pre-resampling weights; but for $0 < \beta < 1$, C is generically a nondegenerate random variable, not almost-surely equal to 1.

Proof. (a) At $\beta = 0$: $S' = \sum_j (w^j)^1 = \sum_j w^j = J$ (unit mean), and $m = \frac{1}{J} \sum_k (w^{p_k})^0 = \frac{1}{J} \sum_k 1 = 1$. So $C = 1 \cdot J/J = 1$, for every realization — this is simply the statement that classical resampling to equal weights removes no likelihood information: the entire mass $S = J$ is preserved exactly, and no renormalization is needed ($m = 1$ trivially).

(b) At $\beta = 1$: $S' = \sum_j (w^j)^0 = J$, and $m = \frac{1}{J} \sum_k (w^{p_k})^1 = \frac{1}{J} \sum_k w^{p_k}$. Since selection at $\beta = 1$ is uniform systematic resampling (every $(w^j)^0 = 1$, so all ancestors are equally likely), and systematic resampling from a uniform base measure over J categories, each with population fraction exactly $1/J$, reproduces each index exactly once (systematic resampling against a perfectly uniform cumulative distribution is a bijection, not merely unbiased): p is a permutation of $1, \dots, J$, hence $m = \frac{1}{J} \sum_k w^{p_k} = \frac{1}{J} \sum_j w^j = 1$ (unit mean, again). So $C = 1 \cdot J/J = 1$, for every realization: at $\beta = 1$ resampling is a relabeling that touches neither the weights nor their sum.

(c) S' is a deterministic function of the entering weights, so the resampling randomness enters only through m . Writing $q_j = (w^j)^{1-\beta}/S'$ for the selection probabilities, $\mathbb{E}[m \mid w^{1:J}] = \sum_j q_j (w^j)^\beta = \frac{1}{S'} \sum_j w^j = J/S'$ (using $\sum_j w^j = J$), so $\mathbb{E}[C \mid w^{1:J}] = (J/S') \cdot (S'/J) = 1$, exactly, for every β — this computation makes no use of $\beta \in \{0, 1\}$ and so subsumes (a) and (b) as the special, degenerate cases in which m (and hence C) is not merely mean-one but almost surely equal to 1. For $0 < \beta < 1$, by contrast, the selection multiplicities are not uniform and m varies with the resampling draw: C is a genuine random variable with mean 1, not the constant 1. $\square$

Corollary 1. *C of Proposition 1 is identically 1 whenever the filter is run at its classical default ($\beta = \text{target} = 0$) or at the fully weighted extreme ($\beta = 1$); at those two settings, restoring C or dropping it makes no difference of any kind, to any realization — multiplying by exactly 1 changes nothing. Only at intermediate β does C ever differ from 1, and it is precisely there, as the next subsection shows, that dropping it costs more than the conditional-mean-one property of part (c) might suggest.*

3.4 Conditional mean one is not enough

The code, as it stands, does not restore C : `systematic_resample!` returned `nothing`, and C was never multiplied into the likelihood. This was precisely the bookkeeping already used, for entirely unrelated reasons of implementation convenience, in the R `pomp` package’s particle filter (`wfilter.c`): weights are carried forward without restoring C at resampling, and each observation time’s conditional log likelihood is instead computed as the log of a *ratio* of successive total masses, `loglik = maxlogw + log(w) - log(oldw)` (`wfilter.c`, line 110), where `oldw` is simply whatever total mass the carried weights happen to have on entry to that step — resampled and renormalized, or not. C is dropped, not restored, by this route too: the mass change across a resampling step is never entered into the likelihood at all, and the next observation’s increment is taken relative to whatever baseline the carried weights land on. Aaron King’s R implementation and, until now, both Julia translations compute, and always have computed, the same estimator, and it is biased.

Proposition 1(c) shows $\mathbb{E}[C \mid w^{1:J}] = 1$ for every β . It is tempting to conclude from this alone that dropping C costs only variance: a factor with conditional expectation one, multiplied into an otherwise-unbiased estimator, cannot change that estimator’s expectation — *if* the factor is independent of everything else the estimator depends on. C is not independent of what it multiplies. $C = mS'/J$ is a function of the realized selections $p_{1:J}$, i.e. of *which ancestors survive resampling*, and those same ancestors are exactly the particles that generate every future contribution to the likelihood: the process draws at every subsequent observation time, and the measurement densities evaluated against them, all descend from $X_n^{p_{1:J}}$. Knowing $\mathbb{E}[C] = 1$ marginally, or even conditionally on the pre-resampling weights, does not give $\mathbb{E}[C \cdot \ell] = \mathbb{E}[\ell]$ for a downstream quantity ℓ that is correlated with C through the shared ancestry — and a resampling step that determines which particles survive is, by construction, correlated with everything those particles subsequently produce. Dropping C therefore biases the likelihood estimate, by $\Theta(1/J)$ per resampling step at

$0 < \beta < 1$; Corollary 1 already confines the effect to that range, since $C \equiv 1$ pins the bias to exactly zero at $\beta \in \{0, 1\}$. Section 3.5 makes this concrete with an exact calculation small enough to enumerate by hand, and then with an end-to-end filtering test.

3.5 An exact enumeration, and an end-to-end test

A two-particle enumeration. Take $J = 2$ particles with unit-mean weights $w = (1.8, 0.2)$ and $\beta = 1/2$. Then $S' = \sqrt{1.8} + \sqrt{0.2} \approx 1.7889$, $q = (0.75, 0.25)$, and two-particle systematic resampling draws the ancestry $(1, 1)$ with probability $1/2$ and $(1, 2)$ with probability $1/2$ (the two possible outcomes of a single systematic sweep against $J = 2$ categories with these selection probabilities). Under $(1, 1)$: $m = \frac{1}{2}(1.8^{1/2} + 1.8^{1/2}) = \sqrt{1.8} \approx 1.3416$ and $C = mS'/2 \approx 1.2$. Under $(1, 2)$: $m = \frac{1}{2}(1.8^{1/2} + 0.2^{1/2}) = S'/2 \approx 0.8944$ and $C \approx 0.8$. (Both values of C round to exactly 1.2 and 0.8; this is not a coincidence of rounding but exact arithmetic, since $\sqrt{1.8} \cdot S'/2$ and $(S'/2) \cdot S'/2$ simplify exactly using $S' = \sqrt{1.8} + \sqrt{0.2}$ and $\sqrt{1.8}\sqrt{0.2} = 0.6$.) Let the future contribution from the two particles be $g = (1, 0)$; the correctly weighted predictive quantity is $\frac{1}{2} \sum_i w_i g_i = 0.9$. Dropping C gives $\frac{1}{2}(1) + \frac{1}{2}(0.75) = 0.875$; restoring it gives $\frac{1}{2}(1.2 \cdot 1) + \frac{1}{2}(0.8 \cdot 0.75) = 0.9$, exactly — this has been verified numerically to six decimal places over 4,000,000 replicates of the resampling step.

An end-to-end test. `test/iid.jl` contains two test sets built around this distinction. The first uses a latent process with no memory (X_t drawn afresh, independently of X_{t-1} , at every step), and documents that such a model is *blind* to the effect: the one-step predictive density is then the same for every particle regardless of which ancestors resampling selects, so the tempered filter is exactly unbiased whatever it does with C , at any number of replicates. The second test constructs the case the first cannot: a frozen binary latent state, $X \sim \text{Bernoulli}(1/2)$ with $X_t = X$ for all t , so the surviving particle after resampling determines the entire remainder of the path. Measurement potentials $g_1(1) = 1.8$, $g_1(0) = 0.2$ and $g_2(1) = 1.0$, $g_2(0) = 0.0$ give an exact likelihood $Z = \mathbb{E}[g_1(X)g_2(X)] = (1/2)(1.8)(1.0) = 0.9$. At $J = 2$ and $\beta = 1/2$ — the setting of the hand calculation above, where the exact predicted value of the dropped- C estimator is 0.8875 — the test measures $\mathbb{E}[\hat{Z}] \approx 0.8879$ over 200,000 replicates, a discrepancy of about 27 Monte Carlo standard errors from the exact $Z = 0.9$; restoring C removes the discrepancy. The same test reports the gap shrinking as J grows, consistent with the $\Theta(1/J)$ scaling of Proposition 1(c).

3.6 The code

`systematic_resample!` now returns $\log C$ rather than `nothing`, and the caller accumulates it:

Listing 2: The end of `systematic_resample!` (`pfilter.jl`, lines 481–490): C computed and returned

```

1      stot::W = s
2      @inbounds for j in eachindex(p)
3          ucum[j] = w[p[j]]^beta
4      end
5      @inbounds for j in eachindex(w)
6          w[j] = ucum[j]
7      end
8      m::W = mean(w)
9      w ./= m # subsequent steps rely on the weights having unit mean
10     log(m*stot/n)
11 end

```

Listing 3: The caller, `pfilt_step_comps!` (`pfiler.jl`, line 315): the return value is accumulated

```
1 logLik[] += systematic_resample!(p, w, work, target)
```

With this bookkeeping, `cond_logLik[k]` — no longer left untouched by resampling — restores the identity $\hat{\mathcal{L}}(\theta) = \prod_n \hat{\mathcal{L}}_n$ as an unbiased estimator of $\mathcal{L}(\theta)$ uniformly over the trigger/target parameter space, by the proper-weighting route of Section 3.2: the swarm handed to the next observation time is, once C is restored, exactly the properly weighted swarm that section derives, not merely a mean-one rescaling of it. Aaron King’s Julia translation and R `pomp`’s `wpfilter.c` still drop C ; Section 7 validates the filter as a whole numerically against an exact reference likelihood, and Section 8 narrates how this correction came to be made twice.

4 ESS-triggered resampling

4.1 Effective sample size

The effective sample size of a weighted particle swarm $w^{1:J}$ is the classical diagnostic

$$\text{ESS}(w^{1:J}) = \frac{\left(\sum_{j=1}^J w^j\right)^2}{\sum_{j=1}^J (w^j)^2},$$

a quantity ranging from 1 (all mass on one particle — maximal weight degeneracy) to J (all weights equal — no degeneracy), invariant to a uniform rescaling of the weights. It is the standard proxy for “how many independent, equally weighted particles is this weighted swarm worth,” and is the basis for adaptive resampling schedules: resample only when the swarm has become sufficiently degenerate to threaten the accuracy of the filtering approximation, rather than at every step regardless of need.

4.2 `compute_ess_logLik!`: the two methods

Two methods of `compute_ess_logLik!` appear in the code (lines 330–403), corresponding to the classic and general engines respectively.

Classic method (lines 330–360), no carried weight. Given only the incremental log weights $\log w_n^j = \log g(y_n \mid X_n^j; \theta)$ (no prior weight carried in, since the classic engine always resamples to equal weights at the previous step), the routine:

- finds $\ell_{\max} = \max_j \log w_n^j$ (line 335) and subtracts it from every log weight (line 344), the standard log-sum-exp stabilization, so that the largest stabilized weight is 1 before exponentiating;
- accumulates $s = \sum_j v^j$ and $ss = \sum_j (v^j)^2$ with $v^j = \exp(\log w_n^j - \ell_{\max})$ (lines 341–348);
- sets $\text{ESS} = s^2/ss$ (line 350), exactly the definition above, computed on the stabilized (equivalently, unstabilized — ESS is scale-invariant) weights;
- computes the conditional log likelihood as $\log \hat{\mathcal{L}}_n = \ell_{\max} + \log(s/J)$ (lines 349–352), the log-sum-exp identity $\log\left(\frac{1}{J} \sum_j w_n^j\right) = \ell_{\max} + \log\left(\frac{1}{J} \sum_j e^{\log w_n^j - \ell_{\max}}\right)$;

- overwrites `logw` with the log weights renormalized to have mean-one linear scale, $\log w_n^j - \log(s/J)$ (line 353), so that *even along the path where resampling does not subsequently occur*, the weights handed onward are on a canonical, likelihood-consistent scale (used, for instance, directly by `systematic_resample!`'s classic sibling method, or, in the general engine, becoming the next step's carried weight).

If $\ell_{\max}$ is not finite (line 340's `else` branch), every particle has zero measurement density — the swarm has been exhausted, filter failure — and the routine reports $\text{ESS} = 0$, $\log \hat{\mathcal{L}}_n = -\infty$ (lines 355–357).

General method (lines 368–403), folding in a carried weight. The general engine carries a linear, unit-mean weight vector `w` forward across observation times whenever resampling does not occur at every step (Section 5); this method folds that carried weight into the current step's incremental log weight before computing ESS and the conditional log likelihood, and — critically — writes the folded, renormalized result *back into* `w` on exit (line 395, `w ./= lik`), maintaining the unit-mean invariant on which every other routine in the filter relies. Concretely:

$$\log \tilde{w}^j = \log w_{\text{carry}}^j + \log w_n^j - \ell_{\max} \quad (\text{line 384}),$$

combining the incoming (unit-mean) carried weight multiplicatively with the new incremental weight, again with the log-sum-exp stabilization $\ell_{\max} = \max_j \log w_n^j$ (the incremental weight's own maximum, not the combined one — since the carried weight already has unit mean and no extreme values by construction). ESS and $\log \hat{\mathcal{L}}_n$ are computed exactly as before from $s = \sum_j \exp(\log \tilde{w}^j)$ and $ss = \sum_j \exp(\log \tilde{w}^j)^2$, with the comment on line 390 flagging explicitly that the formula $\log \hat{\mathcal{L}}_n = \ell_{\max} + \log(s/J)$ is valid here *only because* the carried weight entered with unit mean — the same telescoping-identity fact derived in Section 3. On exit, `w` is renormalized to unit mean (line 395) and `logw` is renormalized to zero mean on the log-linear scale (line 394), so that whichever code path runs next (resampling or not) sees a canonically scaled representation.

4.3 The trigger

The trigger threshold, `trigger` $\in [0, 1]$ (denoted here by the same identifier, since it plays no separate Greek-letter role in the mathematics), governs whether resampling occurs at all: resampling is invoked whenever

$$\text{ESS}_n \leq \text{trigger} \cdot J,$$

tested at line 314 (`ess[] ≤ trigger*n`), *and* the maximum log weight is finite (a degenerate/exhausted swarm always triggers the fallback path, never the resampling path, since there is nothing informative to resample). `trigger = 1` resamples unconditionally, since $\text{ESS}_n \leq J$ always; `trigger = 0` never resamples via this branch (short of exact degeneracy), since $\text{ESS}_n \geq 1 > 0$ always for a non-degenerate swarm.

5 The filtering step

`pfilt_step_comps!` assembles one full observation-time update: weighting statistics, the resampling decision, and (when triggered) the resampling operation itself, dispatched to one of two methods according to the number of arguments supplied — a compile-time dispatch on whether the classic or general engine is in use, resolved once, outside the per-step loop, by the choice made in `pfilter` at lines 98–136 of which `pfilter.internal!` method to call.

5.1 Classic path: trigger = 1, target = 0

Listing 4: Classic filtering step (pfilter.jl, lines 271–293)

```

1 pfilt_step_comps!(
2   logLik::AbstractArray{W,0},
3   ess::AbstractArray{W,0},
4   logw::AbstractArray{W,1},
5   p::AbstractArray{I,1},
6   xp::AbstractArray{X,1},
7   xf::AbstractArray{X,1},
8   work::AbstractArray{W,1},
9   resamp::AbstractArray{Bool,0},
10  n::Integer = length(logw),
11 ) where {W<:AbstractFloat,I<:Integer,X<:NamedTuple} = begin
12   logwmax = compute_ess_logLik!(ess, logLik, logw)
13   if isfinite(logwmax)
14     systematic_resample!(p, logw, work)
15     @inbounds xf .= xp[p]
16     resamp[] = true
17   else
18     p .= collect(eachindex(p))
19     xf .= xp
20     resamp[] = false
21   end
22   nothing
23 end

```

This is the algorithm of Section 1.2 exactly: $\log \hat{\mathcal{L}}_n$ and ESS_n are computed from the incremental weights alone (no carried weight — every step starts from equally weighted particles, since the classic engine resamples unconditionally whenever the swarm is non-degenerate), then classical (untempered, $\beta = 0$) systematic resampling is applied whenever the weights are informative ($\log w_{\max}$ finite), replacing X_n^p for the filtered particles X_n^f (line 285, `xf .= xp[p]`). In the degenerate case, the identity permutation is used and the predicted particles are carried through unchanged as the filtered particles (lines 288–290) — the swarm has failed, but the routine does not error, allowing the caller to detect and report filter failure through $\log \hat{\mathcal{L}}_n = -\infty$ downstream.

5.2 General path: weighted, trigger/target

Listing 5: General filtering step (pfilter.jl, lines 299–324)

```

1 pfilt_step_comps!(
2   logLik::AbstractArray{W,0},
3   ess::AbstractArray{W,0},
4   logw::AbstractArray{W,1},
5   p::AbstractArray{I,1},
6   xp::AbstractArray{X,1},
7   xf::AbstractArray{X,1},
8   w::AbstractArray{W,1},
9   work::AbstractArray{W,1},
10  trigger::W,
11  target::W,
12  resamp::AbstractArray{Bool,0},

```

```

13   n::Integer = length(logw),
14 ) where {W<:AbstractFloat,I<:Integer,X<:NamedTuple} = begin
15   logwmax = compute_ess_logLik!(ess, logLik, logw, w)
16   if isfinite(logwmax) && ess[] <= trigger*n
17     logLik[] += systematic_resample!(p, w, work, target)
18     @inbounds xf .= xp[p]
19     resamp[] = true
20   else
21     p .= collect(eachindex(p))
22     xf .= xp
23     resamp[] = false
24   end
25   nothing
26 end

```

The general step folds the carried weight w into the incremental weight via `compute_ess_logLik!`'s second method (Section 4), then resamples only when both the swarm is non-degenerate *and* the trigger condition $\text{ESS}_n \leq \text{trigger} \cdot J$ is met — otherwise the entire weighted particle representation is carried forward untouched (lines 319–321), with `logLik[]` left at the value set by `compute_ess_logLik!` alone. When resampling does occur, it is tempered resampling at the supplied $\beta = \text{target}$ (line 315); as Section 3 works out in detail, `logLik[]` *is* updated by this call — the normalizing constant C that the resampling routine's own unit-mean renormalization would otherwise remove is computed inside `systematic_resample!` and added into `logLik[]` at this line, restoring the properly weighted representation of Section 3.2 rather than leaving C to be silently absorbed downstream.

Comparing the two methods: setting `trigger = 1` and `target = 0` in the general path reduces it, step for step, to the classic path — $\text{ESS}_n \leq 1 \cdot J$ always holds, so resampling always triggers, and $\beta = 0$ tempered resampling coincides with classical resampling exactly, C of Proposition 1(a) being identically one at $\beta = 0$, so no discrepancy of any kind arises between the two paths at this setting. The classic method is retained separately in the code as a specialized, slightly cheaper implementation of this special case (no carried-weight bookkeeping, no trigger test to evaluate), not because the mathematics differs.

6 Weighted terminal ancestry draw

6.1 The problem

At the end of a filtering run, the package reports a single simulated trajectory $X_{0:N}^*$ from (an approximation to) the smoothing distribution, obtained by tracing the ancestry of one randomly selected terminal particle back through the recorded resampling permutations `perm`. When the terminal particle swarm is *equally* weighted — guaranteed under the classic engine, where every step resamples to equal weights — a uniform draw over the J terminal particles is exactly correct: each terminal particle already represents an equal share of the filtering measure's mass. But whenever `target > 0` or `trigger < 1`, the terminal swarm can carry unequal weights $w^{1:J}$ (accumulated exactly as in Sections 2–4), and a uniform draw over particle *indices* no longer targets the filtering measure: it implicitly reweights every particle to $1/J$, discarding the very information the weights encode.

6.2 A two-particle counterexample

Example 1. Take $J = 2$ terminal particles with weights $w^1 = 3, w^2 = 1$ (unit mean: $\frac{1}{2}(3+1) = 2 \neq 1$ in this raw form, but the argument is unaffected by an overall rescaling — take $w^1 = 1.5, w^2 = 0.5$ for the unit-mean version if preferred). The properly weighted representation of the filtering distribution at this final time is

$$\pi(\cdot) \propto w^1 \delta_{X^1}(\cdot) + w^2 \delta_{X^2}(\cdot) = \frac{3}{4} \delta_{X^1}(\cdot) + \frac{1}{4} \delta_{X^2}(\cdot)$$

after normalizing: particle 1 should be selected as the terminus of the reported trajectory with probability 3/4, particle 2 with probability 1/4. A uniform draw, $j \sim \text{Unif}\{1, 2\}$, instead selects each particle with probability 1/2 — targeting the unweighted empirical measure $\frac{1}{2}\delta_{X^1} + \frac{1}{2}\delta_{X^2}$, a different distribution whenever $w^1 \neq w^2$, and one with no claim to approximating the filtering distribution of interest. The discrepancy is not a higher-order or asymptotically vanishing effect: it persists at every J , since it is a first-moment (bias in the selection probability), not a sampling-variance, error.

This is precisely the defect flagged, as a marginal question rather than a firm claim, against the uniform draw in an earlier version of `trace_ancestry!` (memo, item 4): "I noticed your marginal note there, questioning whether the first index in the ancestry is right — I suspect this is exactly the issue."

6.3 The corrected, five-argument `trace_ancestry!`

Listing 6: Weighted ancestral-lineage tracing (`pfilter.jl`, lines 508–544)

```

1 trace_ancestry!(
2     traj::AbstractArray{X,1},
3     filt::AbstractArray{X,2},
4     perm::AbstractArray{I,2},
5     w::AbstractArray{W,1},
6     work::AbstractArray{W,1},
7 ) where {X,I<:Integer,W<:AbstractFloat} = begin
8     @assert size(traj,1)==size(perm,1)
9     @assert size(filt)==size(perm)
10    @assert length(w)==size(perm,2)
11    n = length(w)
12    wmax::W = -Inf
13    for k in eachindex(w)
14        @inbounds wmax = (w[k] > wmax) ? w[k] : wmax
15    end
16    j::I = if isfinite(wmax)
17        s::W = 0
18        for k in eachindex(w)
19            @inbounds v::W = w[k]
20            s += v
21            @inbounds work[k] = s
22        end
23        u::W = s*rand(LogLik)
24        jj::I = 1
25        while (u > work[jj] && jj < n)
26            jj += 1
27        end

```

```

28     jj
29     else
30         rand(axes(perm,2))
31     end
32     for i in Iterators.reverse(axes(perm,1))
33         @inbounds traj[i] = filt[i,j]
34         @inbounds j = perm[i,j]
35     end
36     j
37 end

```

Cumulative weight construction (lines 523–529). Given the terminal (unit-mean, linear) weight vector $w^{1:J}$, the routine forms the cumulative sums $W_k = \sum_{i=1}^k w^i$, $k = 1, \dots, J$ — `work[k]` — exactly as `systematic_resample!` does for the tempered selection weights, but here for a *single* draw rather than a systematic sweep of J draws.

Weighted single draw (lines 530–535). A draw $u \sim \text{Unif}(0, W_J)$ is made (line 530, `s*rand(LogLik)`), with $W_J = s = \sum_k w^k$, and the smallest index j with $W_j \geq u$ is found by linear search — an inverse-CDF draw from the discrete distribution with $\Pr(J = k) = w^k / \sum_i w^i$, exactly the weighted selection probability required by the properly weighted representation. This is the correction to the counterexample above: with $w^1 = 3, w^2 = 1$, $u \sim \text{Unif}(0, 4)$, and particle 1 is selected whenever $u \in (0, 3]$ — probability $3/4$, as required.

Degenerate fallback (line 537). If every weight is $-\infty$ on the log scale (here checked via a $-\infty$ sentinel on the linear scale, `wmax` not finite — mirroring the fallback convention used throughout `pfilter.jl`), the routine falls back to a uniform draw, matching the equal-weight case and the behavior of the unweighted, four-argument `trace_ancestry!` (lines 483–496) used by the classic engine, where every terminal particle carries equal weight by construction and this method is unnecessary (and unused).

Backward trace (lines 539–543), identical in both methods. Once the terminal index j is fixed (by whichever draw was appropriate), the trajectory is traced backward through the recorded ancestor permutations exactly as in the unweighted case: at each observation time i , from the last to the first, `traj[i] ← filt[i, j]`, then `j ← perm[i, j]`, walking the lineage of ancestors back to time t_0 . This part of the routine is unaffected by weighting — the weighting question is entirely about *which* terminal lineage to initiate, not how to trace one once chosen — and is shared, verbatim in structure, between the two methods.

7 The Gompertz–Kalman validation

7.1 The log-Gompertz linear-Gaussian reduction

The Gompertz population model, in the parametrization used by the package’s built-in example, is

$$X_t = X_{t-1}^S K^{1-S} \varepsilon_t, \quad S = e^{-r}, \quad \varepsilon_t \sim \text{LogNormal}(0, \sigma_p^2),$$

with a lognormal measurement model, $\text{pop}_t \mid X_t \sim \text{LogNormal}(\log X_t, \sigma_m^2)$. Writing $Z_t = \log X_t$, taking logs of the process equation gives an exactly linear, Gaussian autoregression:

$$Z_t = (1 - S) \log K + S Z_{t-1} + w_t, \quad w_t \sim \text{Normal}(0, \sigma_p^2),$$

and the measurement model becomes, on the log scale, another linear-Gaussian observation equation:

$$\log(\text{pop}_t) = Z_t + v_t, \quad v_t \sim \text{Normal}(0, \sigma_m^2).$$

This is a scalar linear-Gaussian state-space model, for which the likelihood of $\log(\text{pop}_{1:N})$ is available exactly, with no Monte Carlo error, via the Kalman filter — a rare and valuable property for validating a Monte Carlo filter implementation, since the comparison is against a deterministic reference rather than a second noisy estimate.

In the package’s built-in Gompertz example, `rinit` sets X_0 to the parameter `X0` deterministically (no process noise at initialization), and the first observation time coincides with t_0 , so no process step intervenes before the first observation is assimilated: the filtering distribution of Z at the first observation time is a point mass, $Z_1 = \log(X_0)$ with zero variance. Every subsequent observation time is preceded by exactly one Gompertz recursion at unit time step, matching the annual spacing of the built-in data; the `discrete_time` step function used by the model ignores its `dt` argument and applies σ_p directly as the per-step log-scale standard deviation, so no $\sqrt{dt}$ rescaling is needed to match the recursion above exactly at $dt = 1$.

7.2 The scalar Kalman recursion

For $t = 1, \dots, N$, with filtering mean and variance m_{t-1}, v_{t-1} of Z_{t-1} given $\log(\text{pop}_{1:t-1})$ (and $m_1 = \log(X_0)$, $v_1 = 0$ at $t = 1$, reflecting the deterministic initialization above):

Predict (skipped at $t = 1$): $m \leftarrow (1 - S) \log K + S m$, $v \leftarrow S^2 v + \sigma_p^2$.

Innovate: forecast variance $F = v + \sigma_m^2$; innovation $e = \log(\text{pop}_t) - m$; contribution to the log likelihood $-\frac{1}{2} (\log(2\pi F) + e^2/F)$.

Update: Kalman gain $\kappa = v/F$; $m \leftarrow m + \kappa e$; $v \leftarrow v - \kappa^2 F$.

Summing the innovation contributions over $t = 1, \dots, N$ gives $\log \mathcal{L}_{\log y}$, the exact conditional log likelihood of $\log(\text{pop}_{1:N})$ under the linear-Gaussian reduction — precisely the computation performed by `kalman_loglik_logy` in `test/gompertz_kalman.jl`.

7.3 The Jacobian correction

The package’s `logdmeasure` for the Gompertz model evaluates the LogNormal density of pop_t itself, not of $\log(\text{pop}_t)$. Since $\text{pop}_t = \exp(\log(\text{pop}_t))$, the two densities are related by the change-of-variables Jacobian $f_{\text{pop}}(y) = f_{\log \text{pop}}(\log y)/y$, so, summing the log Jacobian factor $-\log y_t$ over all observations,

$$\log \mathcal{L}(\text{pop}_{1:N}) = \log \mathcal{L}(\log(\text{pop}_{1:N})) - \sum_{t=1}^N \log(\text{pop}_t).$$

The particle filter, run with the package’s actual `logdmeasure` (a density on the natural, untransformed population scale, as it must be to match the data), estimates $\log \mathcal{L}(\text{pop}_{1:N})$ — the left-hand side — while the Kalman recursion above computes $\log \mathcal{L}_{\log y} = \log \mathcal{L}(\log(\text{pop}_{1:N}))$, the first term on the right. The two must therefore differ by exactly the Jacobian sum, a fixed constant (independent of θ -estimation Monte Carlo error, and indeed independent of the particle filter’s behavior altogether) equal to `jacobian` = $-\sum_t \log y_t$ in the test code.

7.4 Mapping the test

`test/gompertz_kalman.jl` implements exactly this comparison.

Listing 7: Two checks against the exact Kalman likelihood (`test/gompertz_kalman.jl`, lines 89–117, abridged)

```

1 Random.seed!(20260907)
2 Nps = [200,1000,5000]
3 nreps = 30
4 lme = [
5     logmeanexp([logLik(pfilter(P,Np=Np,params=p1)) for _ in 1:nreps],se=true)
6     for Np in Nps
7 ]
8 gap = [x.est - ll_logy for x in lme]
9 se = [x.se for x in lme]
10
11 ## primary check
12 tol_spread = 6*maximum(se)
13 @test maximum(gap)-minimum(gap) < tol_spread
14
15 ## secondary check
16 for (g,s) in zip(gap,se)
17     @test abs(g-jacobian) < 6*s
18 end

```

For each of three particle counts $J \in \{200, 1000, 5000\}$, the classic bootstrap filter (`trigger` = 1, `target` = 0, the package defaults) is run 30 times and the 30 log-likelihood replicates combined via `logmeanexp` (a numerically stable log of an average of exponentials, with a jackknife standard error) into a single estimate `x.est` with reported Monte Carlo standard error `x.se` at that J . `gap` is the difference between this estimate and the exact log-Gompertz-scale Kalman likelihood `ll_logy`, which by the Jacobian identity above should equal `jacobian` up to Monte Carlo error at every J tested — *not* shrink toward zero as J grows.

Primary check. A genuine defect in the filter or in the model definition — as opposed to a mere additive bookkeeping error such as an incorrectly signed Jacobian — would generically make `gap` drift *systematically with J* : the textbook downward bias of the log of a Monte Carlo average of a nonnegative unbiased estimator scales as $-\text{Var}/(2J)$ (a Jensen’s-inequality effect on $\log \hat{\mathcal{L}}$, not on $\hat{\mathcal{L}}$ itself) and shrinks toward zero as $J \rightarrow \infty$; a fixed bookkeeping constant, by contrast, shifts `gap` by the same amount at every J . The primary check tests exactly this distinction: $\max(\text{gap}) - \min(\text{gap})$, the spread of `gap` across the three particle counts, must be small relative to `tol_spread` = $6 \times$ the largest (noisiest, smallest- J) standard error. This test is run only at the classic default settings (`trigger` = 1, `target` = 0), where Proposition 1(a) already guarantees that the resampling bookkeeping question examined in Section 3 — whether the normalizing constant C is restored at resampling, or dropped — cannot make any difference: C is identically one at $\beta = 0$. This check therefore neither tests, nor could have tested, that question one way or the other; Section 3 settles it instead with an exact two-particle enumeration and an end-to-end filtering test built specifically around a latent process with memory, at intermediate β (Section 3.5), since the present check’s model — smooth and well-mixing on the log scale — is close enough to the memoryless case that it could not have detected the bias even had this check been run at $\beta \in (0, 1)$.

Secondary check. `gap` should also be close, in absolute terms, to `jacobian` itself, at every J : this confirms that no fixed bookkeeping error of the kind the primary check is designed to be insensitive to is nonetheless actually present — i.e., that the two checks together pin down both “no systematic drift with J ” and “the correct fixed constant,” leaving no room for an error that happens to be constant in J but wrong in value.

8 History: a normalizing constant lost, retracted, and reinstated

This document did not arrive at Section 3 in one pass. The account there was derived, implemented, then *withdrawn* on the strength of a numerical check, and then reinstated when that check was found to have had no power. The false step is recorded here because what it does and does not show is more instructive than the result itself.

Three candidate defects were raised while mapping each routine in `pfilter.jl` onto its mathematical specification. Two concerned the identity of particles rather than the likelihood. Section 2 found that an earlier translation of the tempered resampling step had indexed the retained weight by selection *slot* rather than by selected *ancestor* — assigning particle k the quantity $(w^k)^\beta$ instead of $(w^{p_k})^\beta$, invisible at $\beta = 0$ but targeting the wrong measure at every $\beta > 0$. That defect was real, and its author found and corrected it independently while this document was being written. Section 6 found its cousin in the terminal ancestry draw: a uniform draw over terminal particle indices where those particles are not, in general, equally weighted. That one remains uncorrected upstream.

The third is the subject of Section 3: the normalizing constant $C = mS'/J$ that unit-mean renormalization removes at every tempered resampling step. It was derived, the one-line change was made — `systematic_resample!` altered to return $\log C$ in place of `nothing`, and `pfilt_step_comps!` to add it into the conditional log likelihood — and then, for a day, it was taken out again.

8.1 The retraction, and why it was wrong

The argument for removing C ran as follows. Proper weighting is a *sufficient* condition for an unbiased sequential Monte Carlo likelihood estimator, not a necessary one, so the failure of that condition does not by itself establish bias; and Proposition 1(c) gives $\mathbb{E}[C \mid w^{1:J}] = 1$ exactly, so C looks like a mean-one factor that a subsequent renormalization will absorb at no cost but variance. A paired numerical comparison seemed to confirm it: on the Gompertz–Kalman reduction of Section 7, with both estimators computed from identical particle draws so that the difference is exactly paired, across $\beta \in \{0.25, 0.5, 0.75\}$ and $J \in \{8, 16, 64\}$ at 400,000 replicates each, the two agreed to within 0.003% of $\hat{\mathcal{L}}$ with no z -score approaching significance.

Both halves of that were mistaken, and they failed independently.

The algebraic half fails at Section 3.4. It is true that $\mathbb{E}[C \mid w^{1:J}] = 1$. It does not follow that $\mathbb{E}[C\ell] = \mathbb{E}[\ell]$, because C is a function of the selected ancestors and is therefore correlated with everything those ancestors go on to generate. A conditionally mean-one factor is harmless only if it is also *independent* of everything downstream, and this one is not. In the two-particle case of Section 3.5, $\mathbb{E}[C] = 1$ exactly while $\text{Cov}(C, \ell) = +0.025$, and $0.875 + 0.025 = 0.9$ accounts for the entire discrepancy.

The empirical half fails for a separate reason, and it is the more general lesson. The bias is $\Theta(1/J)$ per resampling step, and it vanishes identically whenever the one-step predictive density is constant across the cloud. That degeneracy is not exotic: it is exactly what happens when the latent process has no memory, and it is nearly what happens whenever the latent process mixes well, as the Gompertz autoregression does. The paired design could resolve about 0.02% of $\hat{\mathcal{L}}$ against

a bias smaller than 0.15%. The null result was a statement about the experiment, not about the estimator — and it was reported without the standard error that would have made that visible. Rerunning the same comparison with the weight disparity increased recovers the sign the theory requires.

8.2 What each instrument can see

The episode is a clean illustration of three distinct blind spots, and it is worth separating them.

A comparison of *average log likelihoods against an exact reference* is blind to both defects of particle identity. Mislabelling which ancestor’s retained weight lands on which selected particle changes which states carry which weights, and a uniform terminal draw changes which trajectory is reported; neither changes the expected value of the sum of weights at any observation time, which is all the marginal likelihood ever sees. Those two were found only by reading the code against its specification, as Sections 2 and 6 do.

A comparison of *one implementation against another* is blind to any defect the two share. The grid over `trigger` and `target` that agreed cell by cell with R `pomp`’s `wpfilter` was read at the time as validating the tempered resampling beyond the $\beta = 0$ corner. It did not: R keeps its carried weights unnormalized and forms each conditional log likelihood as a ratio of successive total masses, which is algebraically the same estimator as the unit-mean convention with C discarded. Both sides dropped the same factor, so the agreement established that the two implement the same algorithm, not that the algorithm is unbiased.

A comparison against an *exact reference on a model with no power* is blind in the way described above. The Kalman likelihood of Section 7 is genuinely exact and genuinely independent of any implementation; the model it was evaluated on simply could not exhibit the effect at a resolvable magnitude.

What settled the question was none of these, but the exact enumeration of Section 3.5: at $J = 2$ the systematic sweep is driven by a single uniform, so the ancestry is a piecewise-constant function of it and the expectation is a finite sum with no Monte Carlo tolerance at all. The end-to-end test that accompanies it was then built specifically to be *capable* of exhibiting the bias — a frozen latent state, where the surviving particle determines the entire future, which is the exact opposite of the memoryless degeneracy that had concealed it.

8.3 The lesson

Not that term-by-term mathematical accounting is unprofitable: the same method raised all three findings, two of which were real and the third of which was real after all. The lessons are narrower, and both are about how a claim is checked rather than how it is derived.

First: a null numerical result is worth nothing unless the test model is capable of exhibiting the effect and the design has power against it. The standard error belongs beside every null, and the question “could this experiment have detected the thing I am concluding is absent?” belongs before it.

Second: a conditionally mean-one factor is harmless only if it is also independent of everything downstream. $\mathbb{E}[C] = 1$ is not $\mathbb{E}[C\ell] = \mathbb{E}[\ell]$, and the gap between them is exactly $\text{Cov}(C, \ell)$.